0 1 Contents

1 1 Executive Summary

What is MedAI Diagnostics?

An **agentic AI system** mimicking expert clinical reasoning under time pressure. Given a chief complaint, it asks highest-yield questions, maintains a live Bayesian differential, and converges on a grounded diagnosis — all within 60 seconds. Every claim is KB-traceable; hallucination is structurally prevented.

Others	DOCKTOR
Static tree	Bayesian differential
All questions	Max info-gain only
No reasoning chain	Full explainability
No rare diseases	Always seeded
Hallucination-prone	KB-grounded
No feedback loop	Doctor re-trains KB

2 1 System Architecture

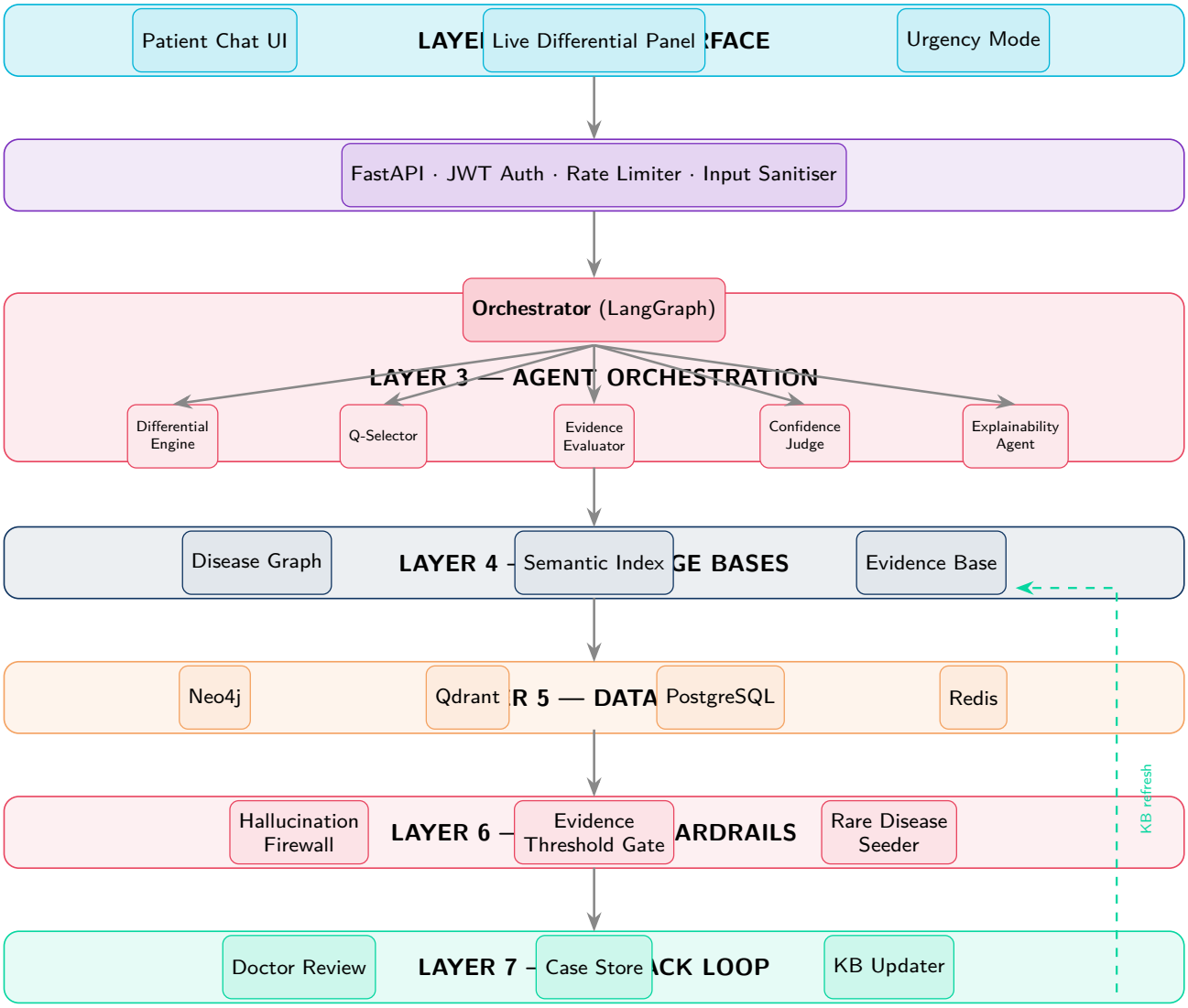


Figure 1: End-to-end MedAI system architecture — seven-layer pipeline

2.1 1Layer explanations

L1 — UI (React + Tailwind). Three simultaneous panels: chat input, live differential with probability bars, and 60-second urgency mode with countdown timer.

L2 — API Gateway (FastAPI). JWT auth, Redis-backed session state, rate limiting, input sanitisation before any agent call.

L3 — Agents (LangGraph + Claude Sonnet 4). Orchestrator drives five sub-agents: *Differential Engine* (Bayesian updater); *Q-Selector* (entropy-reduction question picker); *Evidence Evaluator* (maps answers to Neo4j edges); *Confidence Judge* (gates diagnosis until thresholds met); *Explainability Agent* (grounded reasoning chain).

L4 — Knowledge Bases. Disease Graph (Neo4j ontology), Semantic Index (Qdrant embeddings), Evidence Base (ICD-11, PubMed guidelines).

L5 — Databases. Neo4j (graph), Qdrant (vectors), PostgreSQL (sessions/audit), Redis (hot cache).

L6 — Safety Guardrails. Hallucination Firewall (KB node tracing), Evidence Threshold Gate (min 4 data points), Rare Disease Seeder (floors rare conditions at 2% prior).

L7 — Feedback Loop. Doctor reviews → Case Store → nightly KB Updater → Qdrant re-embed + Neo4j weight adjust.

3 1 Database Design

Design principle

Each engine chosen because the data's *shape* matches its strengths — graph, vector, relational, key-value.

3.0.1 1Neo4j — Disease Knowledge Graph

Stores the medical ontology. Multi-hop relations (`PRESENTS_WITH`, `CONFIRMED_BY`, `RULES_IN`, `RULES_OUT`) are natural graph traversals.

Listing 1: Neo4j schema

```
(:Disease {id,name,icd11_code,prevalence})
(:Symptom {id,name,clinical_term})
(:Test    {id,name,type,normal_range})
(d)-[:PRESENTS_WITH {specificity}]->(s)
(d)-[:CONFIRMED_BY  {weight}]->(t)
(f)-[:RULES_IN  {likelihood_ratio}]->(d)
(f)-[:RULES_OUT {likelihood_ratio}]->(d)
```

3.0.2 1Qdrant — Semantic Symptom Index

Maps patient language to clinical concepts via cosine similarity on 768-dim embeddings. "Chest feels like a heavy weight" maps to `exertional_dyspnoea`.

Listing 2: Qdrant collections

```
Collection: symptoms
  vector_size: 768, distance: Cosine
  payload: {clinical_term, icd_code,
            neo4j_node_id}
Collection: diagnoses
  vector_size: 768, distance: Cosine
  payload: {disease_name, neo4j_node_id}
```

3.0.3 1PostgreSQL — Relational Store

Sessions, accounts, QA history (JSONB), audit trail, doctor corrections.

Listing 3: Key tables

```
CREATE TABLE sessions (
  id          UUID PRIMARY KEY,
  final_dx    TEXT,
  confidence  FLOAT,
  qa_history  JSONB
);
CREATE TABLE doctor_reviews (
  session_id UUID REFERENCES sessions,
  correct_dx TEXT, review_notes TEXT
);
CREATE TABLE audit_log (
  agent TEXT, action TEXT,
  payload JSONB,
  ts TIMESTAMP DEFAULT NOW()
);
```

3.0.4 1Redis — Cache & Session State

Hot differential + QA history in-memory; sub-10ms retrieval avoids re-querying Neo4j/Qdrant every turn.

Key	Value
`session:{id}:diff`	Current differential
`session:{id}:history`	QA pairs
`ratelimit:{ip}`	Request counter
`hot:disease:{id}`	Neo4j subgraph cache

DB	Engine	Ver.	Primary use
Neo4j	Graph	5.x	Disease ontology & relations
Qdrant	Vector	1.9+	Semantic symptom/diagnosis search
PostgreSQL	Relational	16	Sessions, audit, doctor reviews
Redis	In-memory	7.x	Hot cache, session state, rate limits

4 1 Tech Stack

Frontend React 18 · Tailwind · Zustand · Vite

Backend Python 3.12 · FastAPI · Pydantic v2

Agents LangGraph · LangChain · Anthropic SDK

LLM Claude Sonnet 4 (claude-sonnet-4-20250514)

Embeddings OpenAI text-embedding-3-small (768d)

Databases Neo4j · Qdrant · PostgreSQL · Redis

DevOps Docker Compose · GH Actions · AWS ECS

Observability Prometheus · Grafana · Langfuse

Testing pytest · Vitest · Locust · Garak

Why Claude Sonnet 4?

- **Extended thinking** for multi-step diagnostic chains
- **Tool use API** maps cleanly to sub-agent calls
- **Safety training** resists prompt injection

Not fine-tuned — knowledge lives in Neo4j/Qdrant via RAG, keeping it updatable without touching model weights.

Two-engineer split

Partner A	Backend · Agents · Neo4j · Safety
Partner B	Frontend · Infra · PostgreSQL · Redis

5 1 End-to-End Workflow

Pipeline overview

12 sequential steps transform raw patient language into a grounded, explainable, hallucination-checked diagnosis.

1. **Chief complaint entry.** User types symptoms; React sends to FastAPI with session UUID. Redis initialises empty differential + history if new session.
2. **Sanitisation & embedding.** FastAPI strips unsafe chars; `text-embedding-3-small` produces 768-dim vector; Qdrant cosine search maps text to clinical concepts.
3. **Initial differential seeding.** Neo4j Cypher scores diseases by $\sum$specificity, returns top 20. Rare Disease Seeder appends high-stakes conditions at 2% floor. Written to Redis + UI.
4. **Question selection.** Q-Selector calls Claude with current differential; Claude returns the single question maximising entropy reduction over the Neo4j subgraph.
5. **User answers.** Answer goes to Evidence Evaluator, which maps it to `RULES_IN` / `RULES_OUT` Neo4j edges and updates likelihood ratios for every candidate.
6. **Bayesian differential update.** $P(D|E) = \frac{P(E|D)\cdot P(D)}{P(E)}$ using edge-stored likelihood ratios. Scores normalised; Redis + UI updated in real time.
7. **Confidence gate.** Confidence Judge checks: (i) $\geq$4 evidence points, (ii) top diagnosis $P \geq 0.75$, (iii) runner-up $P < 0.40$. Fail $\rightarrow$ step 4. Pass $\rightarrow$ proceed.
8. **Hallucination firewall.** Every claim in top diagnosis traces to a Neo4j node ID. Unresolvable claims stripped before user sees output.
9. **Explainability chain.** Explainability Agent calls Claude with full evidence; output: *"Ruled out X because [evidence]. Confirmed Y because [evidence]. Confidence: [score]."* Grounded text only.
10. **Output rendered.** Diagnosis + confidence + reasoning chain returned to React. Session written to PostgreSQL with QA history in JSONB.
11. **Doctor review (async).** Clinician marks case correct or supplies true diagnosis. Stored in `doctor_reviews`.
12. **KB update (nightly).** Batch job adjusts Neo4j edge weights, re-embeds corrected symptom descriptions, upserts into Qdrant. System improves each cycle.

6 1 Hallucinati Mitigation

Highest-risk failure mode

In a medical context, a hallucinated diagnosis could misdirect treatment. Every design decision makes hallucination *structurally impossible*, not just unlikely.

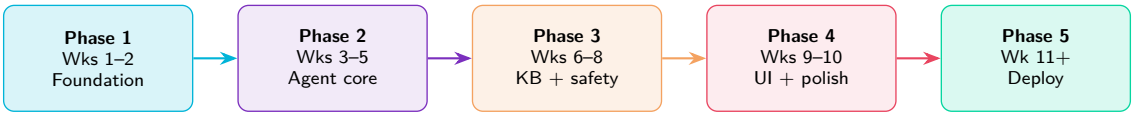
Layer	Mechanism	Prevents
1 Input grounding	Qdrant maps all input to known clinical concepts before LLM sees them	LLM inventing novel symptom terms
2 RAG context	Claude receives Neo4j subgraph as context; reasons over retrieved facts, not weight-encoded memory	Fabricated diagnostic criteria
3 Output verification	Every claim traced to a Neo4j node ID; unresolvable claims stripped	Fabricated evidence in final output

System prompt constraint: *"Only assert diagnoses supported by the provided context. Output `[INSUFFICIENT_EVIDENCE]` if uncertain. Never fabricate test values, prevalence, or criteria. Cite a node ID per statement."*

Garak red-team targets:

- Prompt injection via patient input fields
- Adversarial phrasing to elicit fabricated diagnoses
- Jailbreak attempts to bypass KB-grounding rule

7 1 Execution Plan



Phase 1 — Foundation (Wks 1–2)

- Docker Compose: all 4 DBs + FastAPI running
- FastAPI scaffold: JWT, sessions, Redis integration
- Neo4j schema seeded with 50 common diseases
- Qdrant embedding pipeline bootstrapped
- GitHub repo, branch strategy, CI via GitHub Actions

Phase 2 — Agent Core (Wks 3–5)

- LangGraph orchestrator + state machine
- All 5 sub-agents implemented and wired
- Bayesian update logic in Differential Engine
- Unit tests per agent; E2E on 10 clinical cases

Phase 3 — KB & Safety (Wks 6–8)

- Neo4j expanded: 500+ diseases + ICD-11 codes
- Hallucination Firewall, Evidence Gate, Rare Seeder
- Garak red-team tests; fix critical vulnerabilities
- PostgreSQL audit + doctor review schema finalised

Phase 4 — UI & Polish (Wks 9–10)

- React chat + live differential + 60s urgency mode
- Clinician review dashboard built
- Langfuse LLM observability integrated
- User testing with 3–5 medical students/clinicians

Phase 5 — Deploy (Wk 11+)

- AWS ECS Fargate; separate service per database
- Prometheus + Grafana dashboards live
- Nightly KB update cron (GitHub Actions)
- "Educational tool only" disclaimers throughout UI

8 1 Key Features & Differentiators

Feature	Description
Live differential	Probability bars update every turn
60-sec urgency	Timer; ruthless Q-prioritisation
Semantic entry	Colloquial → clinical via vectors
Explainability	Full ruled-in/out reasoning chain
Rare disease cover	Always seeded; never overlooked
Doctor loop	Corrections re-train KB directly
Grounded outputs	No untraced claim exits the system

Core differentiator

DOCKTOR is neither a symptom checker (rigid tree) nor a raw LLM (hallucination-prone). It is a **grounded agentic reasoning system**: LLM reasoning constrained by a KB firewall, updated by real clinicians, powered by information-theoretic question selection.

- **Entropy-based Q-selection** — no consumer tool does this
- **Updatable KB** — change Neo4j, not model weights
- **Bayesian transparency** — auditable probability distribution
- **Rare disease by design** — seeder, not afterthought

9 1 Risk Register

Risk	Description	Level	Mitigation
Hallucination	LLM invents diagnostic criteria	Med	Firewall + RAG
KB staleness	Medical guidelines change	Low	Nightly update job
Premature Dx	System commits too early	Med	Evidence threshold gate
Rare miss	Uncommon condition never seeded	Low	Rare disease seeder
Prompt inject	Adversarial patient input	Med	Sanitiser + Garak
Liability	Users treat output as real diagnosis	High	"Educational only" UI banner

10 1 Appendix — Environment Setup

Listing 4: Bootstrap commands

```
git clone https://github.com/team/docktor
cd docktor && cp .env.example .env
# Set ANTHROPIC_API_KEY, OPENAI_API_KEY,
#     JWT_SECRET

docker compose up -d
# localhost:8000  FastAPI
# localhost:7474  Neo4j browser
# localhost:6333  Qdrant dashboard
# localhost:5432  PostgreSQL
# localhost:6379  Redis
# localhost:3000  React dev server

python scripts/seed_neo4j.py
python scripts/embed_symptoms.py
pytest tests/ -v
```

Listing 5: requirements.txt (key packages)

```
fastapi>=0.111
pydantic>=2.7
langchain>=0.2
langgraph>=0.1
anthropic>=0.28
openai>=1.30        # embeddings only
neo4j>=5.20
qdrant-client>=1.9
psycopg[binary]>=3.1
redis>=5.0
langfuse>=2.0
garak>=0.9          # LLM safety testing
```